claude-windows / 6a82308f-69d6-43a8-9ea4-d8eaf497253d

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\subagents\workflows\wf_3d536535-241\agent-aa86fa0e87c692ca4.jsonl`
- SHA-256: `c16a955582386c26183a1b377ac596370815b42e043e1d88f9c89b816ca14de7`
- Source modified: `2026-06-16T03:25:30+00:00`
- Imported at: `2026-07-05T16:48:27+00:00`
- project: `wf_3d536535-241`
- session_id: `6a82308f-69d6-43a8-9ea4-d8eaf497253d`

Transcript

1. user (2026-06-16T03:24:24.480Z)

Adversarial Claim Verifier (voter 1/3)

Be SKEPTICAL. Try to REFUTE this claim. ?2/3 refutations kill it.

Research question

Research question: **What concrete, recent (2022?2025) techniques and evaluation practices should a local, commercially-licensed badminton rally-detector adopt to close the quality gap with a strong cloud VLM (Gemini), given a tiny labeled set (~6 ? ~16 golden videos)?** Produce an adversarially-verified, cited report organized around the four dimensions below. Prefer specific named methods, papers (arXiv/venue), and reported numbers over generic advice; flag anything you cannot verify.

CONTEXT (so you target the real gap, not generic ML):
- System: a local pipeline detects the shuttle per-frame (WASB / TrackNet, MIT-licensed) ? a heuristic turns the trajectory into candidate rally windows ? today an optional Gemini "handoff" confirms rallies. We just measured a fully-local (no-AI) run vs Gemini on 3 matches: local emits far FEWER but MUCH LONGER windows (mean 65s vs 6.1s; one window spanned a whole 174s clip) ? it does NOT miss activity, it FAILS TO SEGMENT. Root cause (code-confirmed): a frame is "active" iff the shuttle is visible AND moving above a small per-second-normalized velocity threshold; active frames within a 2 s gap are merged; there is NO max-window cap and NO inactivity/boundary split. So "shuttle is moving" is conflated with "rally in progress," and dead-time + serves + knock-ups bridge rallies into mega-windows. Shuttle tracking itself is excellent (96?99% per-frame visibility).
- HARD CONSTRAINTS: weights/data/code must be MIT/Apache-2.0/BSD only (no viral/NC/custom licenses, reject Gemma-3/Llama-4-MAU-cap); we may NEVER train owned weights on paid-LLM (Gemini/OpenAI/Anthropic) outputs (terms/copyright), though paid LLMs may be used at inference; minors excluded from any training pool; runs on a single local GPU.
- WHAT WE ALREADY HAVE (do NOT re-survey the basics ? go deeper/newer): the architecture skeleton is known to be standard Temporal Action Localization/Segmentation (proposal+scorer) + late/score-level fusion + stacked generalization; in-domain prior art (MonoTrack, ShuttleSet, See et al. 2024/25 non-broadcast badminton rally start/end, Ghosh et al. point-segmentation, tennis/TT play-break state machines) is already cited. We have a working eval harness: leave-one-video-out (grouped by match), temporal-IoU F1 @0.5, F1@{0.1,0.25,0.5}, mAP@tIoU, segment-count-ratio, bootstrap 95% CIs, paired exact sign-test, Platt/isotonic calibration + Brier/ECE, a tiny numpy LogisticRegression scorer over per-window feature columns, an experiment leaderboard, and an "eval?serve" guard (a cue measured +0.074 F1 on permissive proposal windowing but ?0.008 on the coarser served windowing ? reverted). Available but default-OFF per-window signals: net-crossing count (rally gate), 5 person/court features, 3 audio-hit features.

THE FOUR DIMENSIONS:

1) WINDOWING & BOUNDARIES (Temporal Action Localization/Segmentation): beyond proposal+score ? what are the best *recent* methods for (a) turning a dense per-frame signal into well-segmented

actions, (b) explicitly controlling OVER-segmentation/merging, (c) boundary detection/regression and splitting merged segments? Cover e.g. MS-TCN/MS-TCN++, ASRF (action-boundary regression), BMN/BSN boundary-matching, ActionFormer/TriDet/TemporalMaxer (actionness + boundary heads), smoothing/boundary losses, and the standard metrics for over-segmentation (segmental F1@k, edit/Levenshtein score, mAP@tIoU). Which transfer to a 1-D shuttle-trajectory + auxiliary-cue setting on tiny data?

2) SHUTTLE-TRAJECTORY ? RALLY SIGNAL PROCESSING (racket-sports specific): how do published systems convert ball/shuttle tracking into rally/point segments and distinguish "in-play" from dead-time? Cover rally-state machines, serve/let detection, net-crossing & two-sided-occupancy cues, hit/contact (audio + kinematic) detection, and fps-invariant trajectory features. What features best separate "rally" from "shuttle merely moving"?

3) SMALL-DATA STRATEGIES: with ~6?16 labeled videos, what gives the most quality per label ? active learning (which videos/segments to label next; uncertainty/diversity/core-set), semi-/weakly-/point-supervised TAL (e.g. SF-Net, timestamp supervision), self-training/pseudo-labeling, and data-efficient TAL? Separately: the legally-clean ways to exploit a strong teacher (Gemini) at INFERENCE for weak/pseudo-labels or as an evaluation oracle WITHOUT its outputs becoming training data for owned weights ? what does the literature say about teacher-student / distillation boundaries and label-noise handling, and what governance distinctions matter?

4) EVALUATION METHODOLOGY & RIGOR at small n: trustworthy measurement with ~6?16 videos ? bootstrap/permutation tests, paired sign-tests, leave-one-group-out vs RLOOCV bias, calibration (reliability, ECE), the over-segmentation-blindness of plain tIoU, shadow/A-B evaluation, regression gates, and concrete pitfalls (label noise, annotator agreement/IAA, multiple-comparison/over-fitting traps, eval-vs-serve drift). What metrics + protocols should a small-data sports-segmentation leaderboard standardize on?

Deliverable: a structured, cited report with, per dimension, the 3?5 highest-leverage techniques that APPLY to our constrained setting (small data, local, license-clean, 1-D trajectory + cues + learned scorer), each with: what it is, why it fits our over-merge/small-data problem, the expected difficulty/payoff, and the citation. Call out which ideas are low-effort high-confidence wins vs research bets. Be explicit about anything unverified or license-incompatible.

Claim under review

"TemporalMaxer is a single-stage TAL detector built directly on the ActionFormer anchor-free head (same sequence-labeling decoder where every moment is classified plus onset/offset regression via Focal Loss + DIoU loss), with the ONLY change being that ActionFormer's self-attention temporal-context-modeling block is replaced by a parameter-free MaxPooling block (kernel 3, stride 2) between feature-pyramid levels."

Source: https://arxiv.org/abs/2303.09055 (primary)
Supporting quote: "Similar to ActionFormer, TemporalMaxer follows a minimalistic design of sequence labeling where every moment is classified, and their corresponding action boundaries are regressed... The sole variation in our study was the substitution of the Transformer block in ActionFormer with the Max Pooling block."

Checklist

1. Is the claim actually supported by the quote, or is it an overreach/misread?
2. WebSearch for contradicting evidence ? does any credible source dispute or heavily qualify this?
3. Is the source quality sufficient for the claim's strength? (extraordinary claims need primary sources)
4. Is the claim outdated? (check dates ? old claims about fast-moving fields are suspect)
5. Is this a marketing claim / press release / cherry-picked benchmark / forum speculation?

refuted=true if: unsupported by quote / contradicted / low-quality source for strong claim / outdated / marketing fluff.
refuted=false ONLY if: claim is well-supported, current, and source quality matches claim strength.
Default to refuted=true if uncertain.

Structured output only. Evidence MUST be specific.

2. assistant (2026-06-16T03:24:30.692Z)

I'll verify this claim about TemporalMaxer's architecture against the primary source.

3. assistant (2026-06-16T03:24:34.150Z)

I'll fetch the primary source to verify each specific architectural claim.

4. user (2026-06-16T03:24:41.023Z)

I cannot answer your detailed technical questions about TemporalMaxer's architecture based on this webpage content.

The page only provides an abstract, which states: "we introduce TemporalMaxer, which minimizes long-term temporal context modeling while maximizing information from the extracted video clip features with a basic, parameter-free, and local region operating max-pooling block."

This abstract mentions max-pooling replaces complex mechanisms but does not specify:
- Whether ActionFormer's anchor-free head is used
- Loss function types (Focal Loss, DIoU loss, etc.)
- MaxPooling kernel size and stride
- Whether MaxPooling operates between feature-pyramid levels

To answer your questions accurately, you would need to access the full PDF paper (available at `/pdf/2303.09055` on arXiv) rather than just this abstract page.

5. assistant (2026-06-16T03:24:43.431Z)

The abstract confirms the core idea but not the specifics. Let me fetch the full PDF and search for corroborating/contradicting details.

6. user (2026-06-16T03:24:52.763Z)

I've reviewed the provided PDF content, but it contains compressed/encoded streams that don't yield readable architectural details about TemporalMaxer. The document appears to be a technical paper with figures and embedded data, but the text layers are not directly extractable in the format provided.

To accurately answer your five specific questions about:
1. ActionFormer's anchor-free head structure
2. Loss functions (Focal Loss, DIoU loss)
3. The "sole variation" replacing self-attention with MaxPooling
4. MaxPooling kernel/stride specifications
5. MaxPooling application locations

I would need access to the actual readable text content of the TemporalMaxer paper rather than the binary PDF stream data shown here.

Recommendation: Please provide either:
- The paper's plain text or markdown version
- A different format (HTML, extracted text)
- The specific page/section discussing TemporalMaxer's architecture

This will allow me to extract precise quotes (within your 125-character limit) and provide the exact architectural specifications you're seeking.

[Binary content (application/pdf, 1.1MB) also saved to C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\tool-results\webfetch-1781579965172-5ov5x2.pdf]

7. assistant (2026-06-16T03:24:54.649Z)

The PDF was saved locally. Let me read it directly.

8. user (2026-06-16T03:24:55.659Z)

```
pdftoppm failed: Command 'pdftoppm' not found or is in an unsafe location (current directory)
```

9. user (2026-06-16T03:24:59.884Z)

```
[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
ok
```

10. user (2026-06-16T03:25:05.849Z)

```
<persisted-output>
Output too large (34.6KB). Full output saved to: C:\Users\avidu\.claude\projects\C--Users-avidu-
Projects-badminton-highlight-indexer\6a82308f-69d6-43a8-9ea4-d8eaf497253d\tool-
results\bbvgnhw0j.txt

Preview (first 2KB):
==== KW: sole variation ====
]. This setup in- cluded various components such as decoder design d, non-maximum suppression
(NMS) hyper-parameters in the post-processing stage, data augmentation, learning rate, fea- ture
extraction, and the number of feature pyramid level L. The sole variation in our study was the
substitution of the Transformer block in ActionFormer with the Max Pooling block. All
experiments are conducted with a kernel size of 3 for all TCM blocks. The subsequent ablation
will thor- ough analysis of the effects of varyi
---
==== KW: Max Pooling ====
TemporalMaxer: Maximize Temporal Context with only Max Pooling for Temporal Action Localization
Tuan N. Tang, Kwonyoung Kim, Kwanghoon Sohn* School of Electrical and Electronic Engineering
Yonsel University {tuantng, kyk12, khsohn}@yonsei.ac.kr Abstract Temporal Action Localization
(TAL) is a challenging task i
---
] in ActionFormer [60] to model long-term temporal contexts. Our proposed method, termed as
TemporalMaxer, for the ?rst time exploits the potentials of strong features from pretrained 3D
CNN by only utilizing a basic, parameter-free, local operating Max Pooling block. Our proposed
method, the simplest backbone ever for TAL, maintains only the most critical information on
adjacent and local clip embeddings. Combined with the large receptive ?eld of deep networks, the
whole model outperforms other works by a
---
ings that carry the essential information. To this end, we aim to propose simple yet effective
TCM in a straightforward manner. We argue that the TCM can retain the simplest architecture
while maximizing informa- tive features extracted from 3D CNN. Max Pooling [5] is deemed the
most ?tting block for this purpose. Our proposed method, TemporalMaxer, presents the simplest
architecture ever for this task gaining much faster inference speed. Our ?nding suggests that
the pre-trained feature extractor already p
---
...
</persisted-output>
```

11. assistant (2026-06-16T03:25:07.999Z)

I have the key passage. Let me read the full extraction for the remaining specifics (loss functions, kernel/stride, feature pyramid, sequence labeling).

12. user (2026-06-16T03:25:09.311Z)

```
1       ==== KW: sole variation ====
2       ]. This setup in- cluded various components such as decoder design d, non-maximum
suppression (NMS) hyper-parameters in the post-processing stage, data augmentation, learning
rate, fea- ture extraction, and the number of feature pyramid level L. The sole variation in our
study was the substitution of the Transformer block in ActionFormer with the Max Pooling block.
All experiments are conducted with a kernel size of 3 for all TCM blocks. The subsequent
ablation will thor- ough analysis of the effects of varyi
3       ---
```

4 ==== KW: Max Pooling ====
5 TemporalMaxer: Maximize Temporal Context with only Max Pooling for Temporal Action Localization Tuan N. Tang, Kwonyoung Kim, Kwanghoon Sohn* School of Electrical and Electronic Engineering Yonsel University {tuantng, kyk12, khsohn}@yonsei.ac.kr Abstract Temporal Action Localization (TAL) is a challenging task i
6 ---
7] in ActionFormer [60] to model long-term temporal contexts. Our proposed method, termed as TemporalMaxer, for the ?rst time exploits the potentials of strong features from pretrained 3D CNN by only utilizing a basic, parameter-free, local operating Max Pooling block. Our proposed method, the simplest backbone ever for TAL, maintains only the most critical information on adjacent and local clip embeddings. Combined with the large receptive ?eld of deep networks, the whole model outperforms other works by a
8 ---
9 ings that carry the essential information. To this end, we aim to propose simple yet effective TCM in a straightforward manner. We argue that the TCM can retain the simplest architecture while maximizing informa- tive features extracted from 3D CNN. Max Pooling [5] is deemed the most ?tting block for this purpose. Our proposed method, TemporalMaxer, presents the simplest architecture ever for this task gaining much faster inference speed. Our ?nding suggests that the pre-trained feature extractor already p
10 ---
11 40 50 0.5 0.6 0.7 0.8 0.9 1.0 (c) 3.03 1.06 5.17 Clip 1 2 3 3.1 1 5.5 3 1.2 5 3 1 5 transpose 3.03 1.05 5.19 3.1 1.2 5.5 x AvgPooling MaxPooling 0.37 0.3 0.33 0.27 0.38 0.35 0.25 0.35 0.4 Attention Score X (a) Simplified structure of Attention and Max Pooling module (b) Cosine similarity of Attention Score Figure 2. Comparison of effectiveness of Max Pooling over Transformer [51]. (a) When the input clip embeddings are highly similar, given that the value features V are highly similar and the attention s
12 ---
13 .05 5.19 3.1 1.2 5.5 x AvgPooling MaxPooling 0.37 0.3 0.33 0.27 0.38 0.35 0.25 0.35 0.4 Attention Score X (a) Simplified structure of Attention and Max Pooling module (b) Cosine similarity of Attention Score Figure 2. Comparison of effectiveness of Max Pooling over Transformer [51]. (a) When the input clip embeddings are highly similar, given that the value features V are highly similar and the attention score is distributed equally, the Transformer tends to av- erage out the clip embeddings as done in Av
14 ---
15 en the input clip embeddings are highly similar, given that the value features V are highly similar and the attention score is distributed equally, the Transformer tends to av- erage out the clip embeddings as done in Average Pooling. In con- trast, Max Pooling can retain the most crucial information about adjacent clip embeddings and further remove redundancy in the video. (b) Cosine similarity matrix(bottom-left) of value features in self-attention in ActionFormer [60] where the red boxes exhibit the act
16 ---
17 t (2) Architecture overview. The overall architecture is de- picted in Fig. 3. Our proposed method aims to learn to Pre-trained 3D CNN Feature projection layer ... ... TemporalMaxer TemporalMaxer Cls + Reg Head Cls + Reg Head Cls + Reg Head Max Pooling Sliding Direction ..... TemporalMaxer ..... TemporalMaxer ... ... Figure 3. Overview of TemporalMaxer. The proposed method utilizes Max Pooling as a Temporal Context Modeling block applied between temporal feature pyramid levels to
18 ---
19 ... ... TemporalMaxer TemporalMaxer Cls + Reg Head Cls + Reg Head Cls + Reg Head Max Pooling Sliding Direction ..... TemporalMaxer ..... TemporalMaxer ... ... Figure 3. Overview of TemporalMaxer. The proposed method utilizes Max Pooling as a Temporal Context Modeling block applied between temporal feature pyramid levels to maximize informative features of high similarity clip embedding. Speci?cally, it ?rst extracts features of every clip using pre-trained 3D CNN. After that, the b
20 ---
21 etwork layer, followed by Layer Normalization [3], and ReLU [1]. We simply assign Z1 = Xp as the ?rst feature in Z. Finally, the multi-scale temporal feature pyramid Z is encoded by TemporalMaxer: Zl = TemporalMaxer(Zl?1). (4) Here: TemporalMaxer is Max Pooling and employed with stride 2, Zl ?R T 2l?1 ×D, 2 <= l <= L. It is worth not- ing that ActionFormer [60] employs Transformer [51] as a TCM block where each clip feature at the moment t repre- sents a token, our proposed method adopts only Max Pool- ing
22 ---
23 hyper-parameters in the post-processing stage, data augmentation, learning rate, fea- ture extraction, and the number of feature pyramid level L. The sole variation in our study was the substitution of the Transformer block in ActionFormer with the Max Pooling block. All

experiments are conducted with a kernel size of 3 for all TCM blocks. The subsequent ablation will thor- ough analysis of the effects of varying kernel sizes. During training, the input feature length is kept constant at 2304, correspondi
---
dataset, and the results [10, 48, 9] are taken from [47]. 4.5. Ablation Study We perform various ablation studies to verify the effec- tiveness of TemporalMaxer. To better understand what is the effective component for TCM, we gradually replace the Max Pooling with other blocks such as convolution, sub- sampling, and Average Pooling. Furthermore, we evaluate numerous kernel sizes of Max Pooling. Note that all exper- iments in this section are conducted on the train and valida- tion set of the THUMOS14 dat
---
iveness of TemporalMaxer. To better understand what is the effective component for TCM, we gradually replace the Max Pooling with other blocks such as convolution, sub- sampling, and Average Pooling. Furthermore, we evaluate numerous kernel sizes of Max Pooling. Note that all exper- iments in this section are conducted on the train and valida- tion set of the THUMOS14 dataset. Effective of TemporalMaxer. Tab. 5 presents the re- sults of other blocks other than Max Pooling. Motivated by PoolFormer [58] that
---
evaluate numerous kernel sizes of Max Pooling. Note that all exper- iments in this section are conducted on the train and valida- tion set of the THUMOS14 dataset. Effective of TemporalMaxer. Tab. 5 presents the re- sults of other blocks other than Max Pooling. Motivated by PoolFormer [58] that replaces the computationally in- tensive and highly parameterized attention module with the most basic block in deep learning, the pooling layer. Our studies started by ?rst questioning the most straightforward app
---
t clips is reduced. That is why Average Pooling decreases by 3.6% mAP compared with Transformer. The ablation study with Average Pooling suggests that the most important information of clip embeddings should be maintained. For that reason, we employ Max Pooling as TCM. The result is provided in the last row of Tab. 5. Max Pooling achieves the highest results at every tIoU threshold and signi?cantly outperforms the strong and robust base- line, ActionFormer. To clarify, TemporalMaxer effectively highlights
---
==== KW: MaxPool ====
cs.CV] 16 Mar 2023 Input video Backbone Head Pretrained 3D CNN Input video Transformer Head Pretrained 3D CNN Input video TemporalMaxer Head Pretrained 3D CNN Input video Convolution Head Pretrained 3D CNN Common Architecture AFSD ActionFormer Ours MaxPooling Clip embeddings Input video Graph Head Pretrained 3D CNN G-TAD Figure 1. Common architecture in Temporal Action Localization (TAL) and different temporal context modeling (TCM) blocks. Existing works have incorporated extensive parameters, high c
---
). Re- cent studies have emphasized the necessity of long-term 0 10 20 30 40 50 0.15 0.09 0.12 0.06 0.03 0 10 20 30 40 50 0.5 0.6 0.7 0.8 0.9 1.0 (c) 3.03 1.06 5.17 Clip 1 2 3 3.1 1 5.5 3 1.2 5 3 1 5 transpose 3.03 1.05 5.19 3.1 1.2 5.5 x AvgPooling MaxPooling 0.37 0.3 0.33 0.27 0.38 0.35 0.25 0.35 0.4 Attention Score X (a) Simplified structure of Attention and Max Pooling module (b) Cosine similarity of Attention Score Figure 2. Comparison of effectiveness of Max Pooling over Transformer [51]. (a) Wh
---
==== KW: max-pooling ====
sophisticated architectures. To this end, we introduce TemporalMaxer, which minimizes long-term tem- poral context modeling while maximizing information from the extracted video clip features with a basic, parameter- free, and local region operating max-pooling block. Pick- ing out only the most critical information for adjacent and local clip embeddings, this block results in a more ef?- cient TAL model. We demonstrate that TemporalMaxer out- performs other state-of-the-art methods that utilize long- term
---
==== KW: max pooling ====
TemporalMaxer: Maximize Temporal Context with only Max Pooling for Temporal Action Localization Tuan N. Tang, Kwonyoung Kim, Kwanghoon Sohn* School of Electrical and Electronic Engineering Yonsel University {tuantng, kyk12, khsohn}@yonsei.ac.kr Abstract Temporal Action Localization (TAL) is a challenging task i
---
] in ActionFormer [60] to model long-term temporal contexts. Our proposed method, termed as TemporalMaxer, for the ?rst time exploits the potentials of strong features from pretrained 3D CNN by only utilizing a basic, parameter-free, local operating Max Pooling block. Our proposed method, the simplest backbone ever for TAL, maintains only the most critical

information on adjacent and local clip embeddings. Combined with the large receptive ?eld of deep networks, the whole model outperforms other works by a
45 ---
46 ings that carry the essential information. To this end, we aim to propose simple yet effective TCM in a straightforward manner. We argue that the TCM can retain the simplest architecture while maximizing informa- tive features extracted from 3D CNN. Max Pooling [5] is deemed the most ?tting block for this purpose. Our proposed method, TemporalMaxer, presents the simplest architecture ever for this task gaining much faster inference speed. Our ?nding suggests that the pre-trained feature extractor already p
47 ---
48 40 50 0.5 0.6 0.7 0.8 0.9 1.0 (c) 3.03 1.06 5.17 Clip 1 2 3 3.1 1 5.5 3 1.2 5 3 1 5 transpose 3.03 1.05 5.19 3.1 1.2 5.5 x AvgPooling MaxPooling 0.37 0.3 0.33 0.27 0.38 0.35 0.25 0.35 0.4 Attention Score X (a) Simplified structure of Attention and Max Pooling module (b) Cosine similarity of Attention Score Figure 2. Comparison of effectiveness of Max Pooling over Transformer [51]. (a) When the input clip embeddings are highly similar, given that the value features V are highly similar and the attention s
49 ---
50 .05 5.19 3.1 1.2 5.5 x AvgPooling MaxPooling 0.37 0.3 0.33 0.27 0.38 0.35 0.25 0.35 0.4 Attention Score X (a) Simplified structure of Attention and Max Pooling module (b) Cosine similarity of Attention Score Figure 2. Comparison of effectiveness of Max Pooling over Transformer [51]. (a) When the input clip embeddings are highly similar, given that the value features V are highly similar and the attention score is distributed equally, the Transformer tends to av- erage out the clip embeddings as done in Av
51 ---
52 en the input clip embeddings are highly similar, given that the value features V are highly similar and the attention score is distributed equally, the Transformer tends to av- erage out the clip embeddings as done in Average Pooling. In con- trast, Max Pooling can retain the most crucial information about adjacent clip embeddings and further remove redundancy in the video. (b) Cosine similarity matrix(bottom-left) of value features in self-attention in ActionFormer [60] where the red boxes exhibit the act
53 ---
54 t (2) Architecture overview. The overall architecture is de- picted in Fig. 3. Our proposed method aims to learn to Pre-trained 3D CNN Feature projection layer ... ... TemporalMaxer TemporalMaxer Cls + Reg Head Cls + Reg Head Cls + Reg Head Max Pooling Sliding Direction ..... TemporalMaxer ..... TemporalMaxer ... ... Figure 3. Overview of TemporalMaxer. The proposed method utilizes Max Pooling as a Temporal Context Modeling block applied between temporal feature pyramid levels to
55 ---
56 ... ... TemporalMaxer TemporalMaxer Cls + Reg Head Cls + Reg Head Cls + Reg Head Max Pooling Sliding Direction ..... TemporalMaxer ..... TemporalMaxer ... ... Figure 3. Overview of TemporalMaxer. The proposed method utilizes Max Pooling as a Temporal Context Modeling block applied between temporal feature pyramid levels to maximize informative features of high similarity clip embedding. Speci?cally, it ?rst extracts features of every clip using pre-trained 3D CNN. After that, the b
57 ---
58 etwork layer, followed by Layer Normalization [3], and ReLU [1]. We simply assign Z1 = Xp as the ?rst feature in Z. Finally, the multi-scale temporal feature pyramid Z is encoded by TemporalMaxer: Zl = TemporalMaxer(Zl?1). (4) Here: TemporalMaxer is Max Pooling and employed with stride 2, Zl ?R T 2l?1 ×D, 2 <= l <= L. It is worth not- ing that ActionFormer [60] employs Transformer [51] as a TCM block where each clip feature at the moment t repre- sents a token, our proposed method adopts only Max Pool- ing
59 ---
60 hyper-parameters in the post-processing stage, data augmentation, learning rate, fea- ture extraction, and the number of feature pyramid level L. The sole variation in our study was the substitution of the Transformer block in ActionFormer with the Max Pooling block. All experiments are conducted with a kernel size of 3 for all TCM blocks. The subsequent ablation will thor- ough analysis of the effects of varying kernel sizes. During training, the input feature length is kept constant at 2304, correspondi
61 ---
62 dataset, and the results [10, 48, 9] are taken from [47]. 4.5. Ablation Study We perform various ablation studies to verify the effec- tiveness of TemporalMaxer. To better understand what is the effective component for TCM, we gradually replace the Max Pooling with other blocks such as convolution, sub- sampling, and Average Pooling. Furthermore, we evaluate numerous kernel sizes of Max Pooling. Note that all exper- iments in this section are conducted on the train and valida- tion set of the THUMOS14 dat

63 ---
64 iveness of TemporalMaxer. To better understand what is the effective component for TCM, we gradually replace the Max Pooling with other blocks such as convolution, sub- sampling, and Average Pooling. Furthermore, we evaluate numerous kernel sizes of Max Pooling. Note that all exper- iments in this section are conducted on the train and valida- tion set of the THUMOS14 dataset. Effective of TemporalMaxer. Tab. 5 presents the re- sults of other blocks other than Max Pooling. Motivated by PoolFormer [58] that
65 ---
66 evaluate numerous kernel sizes of Max Pooling. Note that all exper- iments in this section are conducted on the train and valida- tion set of the THUMOS14 dataset. Effective of TemporalMaxer. Tab. 5 presents the re- sults of other blocks other than Max Pooling. Motivated by PoolFormer [58] that replaces the computationally in- tensive and highly parameterized attention module with the most basic block in deep learning, the pooling layer. Our studies started by ?rst questioning the most straightforward app
67 ---
68 t clips is reduced. That is why Average Pooling decreases by 3.6% mAP compared with Transformer. The ablation study with Average Pooling suggests that the most important information of clip embeddings should be maintained. For that reason, we employ Max Pooling as TCM. The result is provided in the last row of Tab. 5. Max Pooling achieves the highest results at every tIoU threshold and signi?cantly outperforms the strong and robust base- line, ActionFormer. To clarify, TemporalMaxer effectively highlights
69 ---
70 ==== KW: Focal ====
71 Note that all the weights of the decoder are shared between the different features in the multi-scale feature pyramid Z. Learning Objective. The model predicts ??t = (os t, oe t, ct) for every moment of the input X. Following the baseline [60], the Focal Loss [31] and DIoU loss [64] are employed to supervise classi?cation and regression outputs respectively. The overall loss function is de?ned as: Ltotal = X t (Lcls + 1ctLreg) /T+ (7) where Lreg denotes regression loss and is applied only when the i
72 ---
73 Ming Yang. Bsn: Boundary sensitive network for temporal action proposal generation. In Proceedings of the European conference on computer vision (ECCV), pages 3?19, 2018. [31] Tsung- Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Doll´ar. Focal loss for dense object detection. In Pro- ceedings of the IEEE international conference on computer vision, pages 2980?2988, 2017. [32] Qinying Liu and Zilei Wang. Progressive boundary re?ne- ment network for temporal action detection. In Proceedings
74 ---
75 ==== KW: DIoU ====
76 eights of the decoder are shared between the different features in the multi-scale feature pyramid Z. Learning Objective. The model predicts ??t = (os t, oe t, ct) for every moment of the input X. Following the baseline [60], the Focal Loss [31] and DIoU loss [64] are employed to supervise classi?cation and regression outputs respectively. The overall loss function is de?ned as: Ltotal = X t (Lcls + 1ctLreg) /T+ (7) where Lreg denotes regression loss and is applied only when the indicator function,
77 ---
78 ==== KW: GIoU ====
79 ==== KW: sequence labeling ====
80 rs as a long-term TCM. Our approach, TemporalMaxer, belongs to the single- stage TAL model that utilizes a state-of-the-art Action- Former [60] as the baseline for comparison. Similar to Ac- tionFormer, TemporalMaxer follows a minimalistic design of sequence labeling where every moment is classi?ed, and their corresponding action boundaries are regressed. The main difference is that we avoid exhausting attention be- tween clips from long-term timeframes which can uninten- tionally ?atten minute information among
81 ---
82 entation [28, 60] for an action instance. Each moment is classi?ed into the background or one of C categories, and regressed the onset and offset based on the current time step of that moment. Consequently, the predic- tion in TAL is formulated as a sequence labeling problem. X = {x1, x2, . . . , xT } ??? = n ??1, ??2, . . . , ??T o (1) At the time step t, the output ??t = (os t, oe t, ct) is de?ned as following: ? os t > 0 and oe t > 0 represent the temporal intervals between the current time step t and the ons
83 ---
84 orth not- ing that ActionFormer [60] employs Transformer [51] as a TCM block where each clip feature at the moment t repre- sents a token, our proposed method adopts only Max Pool- ing [5] as TCM block. Decoder Design The decoder d learns to predict sequence labeling, ?? = n ??1, ??2, . . . , ??T o , for ev- ery moment using multi-scale feature pyramid Z = {Z1, Z2, ..., ZL}.

The decoder adopts a lightweight con- volutional neural network and consists of classi?cation and regression heads. Formally, the two heads
85 ---
86 ==== KW: sequence-labeling ====
87 ==== KW: anchor-free ====
88 ion in a single pass, without using a separate proposal generation step. The pioneering work [40] developed anchor-based single-stage TAL using convolutional networks, inspired by a single- stage object detector [42, 33]. Meanwhile, [28] proposed an anchor-free single-stage model with a saliency-based re- ?nement module. Long-term Temporal Context Modeling (TCM). Re- cent studies have emphasized the necessity of long-term 0 10 20 30 40 50 0.15 0.09 0.12 0.06 0.03 0 10 20 30 40 50 0.5 0.6 0.7 0.8 0.9 1.0 (c
89 ---
90 , en, and an are starting time, ending time, and associated action label an respectively, sn ?[1, T], en ?[1, T], sn < en, and the action label an belongs to the pre-de?ned set of C categories. 3.2. TemporalMaxer Action Representation. We follow the anchor-free single-stage representation [28, 60] for an action instance. Each moment is classi?ed into the background or one of C categories, and regressed the onset and offset based on the current time step of that moment. Consequently, the predic- tion in TAL
91 ---
92 ex Gorban, Ivan Laptev, Rahul Sukthankar, and Mubarak Shah. The thumos challenge on action recognition for videos ?in the wild?. Computer Vision and Image Understanding, 155:1? 23, 2017. [23] Tae-Kyung Kang, Gun-Hee Lee, and Seong-Whan Lee. Ht- net: Anchor-free temporal action localization with hierarchi- cal transformers. In 2022 IEEE International Conference on Systems, Man, and Cybernetics (SMC), pages 365?370. IEEE, 2022. [24] Will Kay, Joao Carreira, Karen Simonyan, Brian Zhang, Chloe Hillier, Sudheen
93 ---
94 regularization. In Computer Vision?ECCV 2020: 16th European Conference, Glasgow, UK, August 23?28, 2020, Proceedings, Part VIII 16, pages 539?555. Springer, 2020. [63] Peisen Zhao, Lingxi Xie, Ya Zhang, and Qi Tian. Actionness-guided transformer for anchor-free temporal ac- tion localization. IEEE Signal Processing Letters, 29:194? 198, 2021. [64] Zhaohui Zheng, Ping Wang, Wei Liu, Jinze Li, Rongguang Ye, and Dongwei Ren. Distance-iou loss: Faster and better learning for bounding box regression. In Proceed
95 ---
96 ==== KW: anchor free ====
97 ==== KW: feature pyramid ====
98 Cls + Reg Head Max Pooling Sliding Direction ..... TemporalMaxer ..... TemporalMaxer ... ... Figure 3. Overview of TemporalMaxer. The proposed method utilizes Max Pooling as a Temporal Context Modeling block applied between temporal feature pyramid levels to maximize informative features of high similarity clip embedding. Speci?cally, it ?rst extracts features of every clip using pre-trained 3D CNN. After that, the backbone encodes clip features to form a multi-scale feature pyramid. The backb
99 ---
100 n temporal feature pyramid levels to maximize informative features of high similarity clip embedding. Speci?cally, it ?rst extracts features of every clip using pre-trained 3D CNN. After that, the backbone encodes clip features to form a multi-scale feature pyramid. The backbone consists of 1D convolutional layers and TemporalMaxer layers. Finally, a lightweight classi?cation and regression head decodes the feature pyramid to action candidates for every input moment. label every input moment by f (X) ??? with
101 ---
102 ession head. e : X ?Z learn to encode the input video feature X into latent vector Z, and d : Z ??? learns to predict labels for every input moment. To effectively capture ac- tions transpiring at various temporal scales, we also adopt a multi-scale feature pyramid representation, which is de- noted as Z = {Z1, Z2, ..., ZL}. Encoder design The input feature X is ?rst en- coded into multi-scale temporal feature pyramid Z = {Z1, Z2, ..., ZL} using encoder e. The encoder e simply contains two 1D convolutional neu
103 ---
104 ctively capture ac- tions transpiring at various temporal scales, we also adopt a multi-scale feature pyramid representation, which is de- noted as Z = {Z1, Z2, ..., ZL}. Encoder design The input feature X is ?rst en- coded into multi-scale temporal feature pyramid Z = {Z1, Z2, ..., ZL} using encoder e. The encoder e simply contains two 1D convolutional neural network layers as fea- ture projection layers, followed by L ?1 Temporal Con- text Modeling (TCM) blocks to produce feature pyramid Z. Formally, the fea
105 ---
106 multi-scale temporal feature pyramid Z = {Z1, Z2, ..., ZL} using encoder e. The encoder e simply contains two 1D convolutional neural network layers as fea- ture projection layers,

followed by L ?1 Temporal Con- text Modeling (TCM) blocks to produce feature pyramid Z. Formally, the feature projection layers are described as: Xp = E2(E1(Concat(X))) (3) The input video feature sequence X = {x1, x2, ..., xT }, where xi ?R1×Din, is ?rst concatenated in the ?rst di- mension and then fed into two feature projection
107 ---
108 ×D with D-dimensional feature space. Each projection module comprises one 1D-convolutional neural network layer, followed by Layer Normalization [3], and ReLU [1]. We simply assign Z1 = Xp as the ?rst feature in Z. Finally, the multi-scale temporal feature pyramid Z is encoded by TemporalMaxer: Zl = TemporalMaxer(Zl?1). (4) Here: TemporalMaxer is Max Pooling and employed with stride 2, Zl ?R T 2l?1 ×D, 2 <= l <= L. It is worth not- ing that ActionFormer [60] employs Transformer [51] as a TCM block where each
109 ---
110 lip feature at the moment t repre- sents a token, our proposed method adopts only Max Pool- ing [5] as TCM block. Decoder Design The decoder d learns to predict sequence labeling, ?? = n ??1, ??2, . . . , ??T o , for ev- ery moment using multi-scale feature pyramid Z = {Z1, Z2, ..., ZL}. The decoder adopts a lightweight con- volutional neural network and consists of classi?cation and regression heads. Formally, the two heads are de?ned as: Cl = Fc(E4(E3(Zl))) (5) Ol = ReLU(Fo(E6(E5(Zl)))) (6) Here, Zl ? R T 2l
111 ---
112 {0, 2l?1, ..., T}. E denotes the 1D convolution followed by Layer Normal- ization and ReLU activation function. Fc and Fo are both 1D convolution. Note that all the weights of the decoder are shared between the different features in the multi-scale feature pyramid Z. Learning Objective. The model predicts ??t = (os t, oe t, ct) for every moment of the input X. Following the baseline [60], the Focal Loss [31] and DIoU loss [64] are employed to supervise classi?cation and regression outputs respectively. The ov
113 ---
114 nly when the indicator function, 1ct, indicates that the current time step t is a positive sample. T+ is the number of positive samples. Lcls is C way classi?cation loss. The loss function Ltotal is applied to all levels on the output of multi-scale feature pyramid Z and averaged across all video samples during training. 4. Experimental Results In this section, we show that our proposed method, Tem- poralMaxer, demonstrates the outstanding results achieved across a variety of challenging datasets, namely THUMO
115 ---
116 eline model, ActionFormer [60]. This setup in- cluded various components such as decoder design d, non-maximum suppression (NMS) hyper-parameters in the post-processing stage, data augmentation, learning rate, fea- ture extraction, and the number of feature pyramid level L. The sole variation in our study was the substitution of the Transformer block in ActionFormer with the Max Pooling block. All experiments are conducted with a kernel size of 3 for all TCM blocks. The subsequent ablation will thor- ough anal
117 ---
118 ==== KW: kernel ====
119 mentation, learning rate, fea- ture extraction, and the number of feature pyramid level L. The sole variation in our study was the substitution of the Transformer block in ActionFormer with the Max Pooling block. All experiments are conducted with a kernel size of 3 for all TCM blocks. The subsequent ablation will thor- ough analysis of the effects of varying kernel sizes. During training, the input feature length is kept constant at 2304, corresponding to approximately 5 minutes of video on both THUM
120 ---
121 our study was the substitution of the Transformer block in ActionFormer with the Max Pooling block. All experiments are conducted with a kernel size of 3 for all TCM blocks. The subsequent ablation will thor- ough analysis of the effects of varying kernel sizes. During training, the input feature length is kept constant at 2304, corresponding to approximately 5 minutes of video on both THUMOS14 and MultiTHUMOS datasets, roughly 20 min- utes on the EPIC-Kitchens 100 dataset, and approximately 45 minut
122 ---
123 ify the effec- tiveness of TemporalMaxer. To better understand what is the effective component for TCM, we gradually replace the Max Pooling with other blocks such as convolution, sub- sampling, and Average Pooling. Furthermore, we evaluate numerous kernel sizes of Max Pooling. Note that all exper- iments in this section are conducted on the train and valida- tion set of the THUMOS14 dataset. Effective of TemporalMaxer. Tab. 5 presents the re- sults of other blocks other than Max Pooling. Motivated by
124 ---
125 3D CNN are informative and is the potential for TAL. However, subsampling is prone to losing the most cru- cial information as only half of the video clip embeddings are kept after a TCM block. Thus, we replace Transformer with Average Pooling with kernel size 3 and stride 2. As

expected, Average Pooling improves the result by 2.2 % mAP on average compared with the subsampling. It is because this operation does not drop any clip embedding which helps to retain the crucial information. However, the Av
126 ---
127 ActionFormer, with 2.8x fewer GMACs and 3x faster inference speed. Especially, when comparing only the backbone time, our proposed method only takes 2.5 ms which is incredibly 8.0x faster than ActionFormer backbone [63], 20.1 ms. Different Values of kernel size. We make an ablation with the kernel size of TemporalMaxer 3, 4, 5, 6 and the average mAPs are 67.7, 67.1, 66.8, 65.7, respectively. Our model achieved the highest performance with a kernel size of 3, while the lowest performance was observed w
128 ---
129 faster inference speed. Especially, when comparing only the backbone time, our proposed method only takes 2.5 ms which is incredibly 8.0x faster than ActionFormer backbone [63], 20.1 ms. Different Values of kernel size. We make an ablation with the kernel size of TemporalMaxer 3, 4, 5, 6 and the average mAPs are 67.7, 67.1, 66.8, 65.7, respectively. Our model achieved the highest performance with a kernel size of 3, while the lowest performance was observed with a ker- nel size of 6. This decrease in
130 ---
131 tionFormer backbone [63], 20.1 ms. Different Values of kernel size. We make an ablation with the kernel size of TemporalMaxer 3, 4, 5, 6 and the average mAPs are 67.7, 67.1, 66.8, 65.7, respectively. Our model achieved the highest performance with a kernel size of 3, while the lowest performance was observed with a ker- nel size of 6. This decrease in performance can be attributed to the corresponding loss of information during training. The results obtained for kernel sizes 3, 4, and 5 are not very s
132 ---
133 d the highest performance with a kernel size of 3, while the lowest performance was observed with a ker- nel size of 6. This decrease in performance can be attributed to the corresponding loss of information during training. The results obtained for kernel sizes 3, 4, and 5 are not very sensitive to the kernel size. This suggests that these kernel sizes can effectively capture the relevant temporal informa- tion for the task at hand. 5. Conclusion In this paper, we propose an extremely simpli?ed ap- p
134 ---
135 e the lowest performance was observed with a ker- nel size of 6. This decrease in performance can be attributed to the corresponding loss of information during training. The results obtained for kernel sizes 3, 4, and 5 are not very sensitive to the kernel size. This suggests that these kernel sizes can effectively capture the relevant temporal informa- tion for the task at hand. 5. Conclusion In this paper, we propose an extremely simpli?ed ap- proach, TemporalMaxer for temporal action localization t
136 ---
137 with a ker- nel size of 6. This decrease in performance can be attributed to the corresponding loss of information during training. The results obtained for kernel sizes 3, 4, and 5 are not very sensitive to the kernel size. This suggests that these kernel sizes can effectively capture the relevant temporal informa- tion for the task at hand. 5. Conclusion In this paper, we propose an extremely simpli?ed ap- proach, TemporalMaxer for temporal action localization task. To minimize the structure, we exp
138 ---
139 ==== KW: stride ====
140 r Normalization [3], and ReLU [1]. We simply assign Z1 = Xp as the ?rst feature in Z. Finally, the multi-scale temporal feature pyramid Z is encoded by TemporalMaxer: Zl = TemporalMaxer(Zl?1). (4) Here: TemporalMaxer is Max Pooling and employed with stride 2, Zl ?R T 2l?1 ×D, 2 <= l <= L. It is worth not- ing that ActionFormer [60] employs Transformer [51] as a TCM block where each clip feature at the moment t repre- sents a token, our proposed method adopts only Max Pool- ing [5] as TCM block. Decode
141 ---
142 - mance on testing videos. Feature Extraction. Following [60, 62], we extract the features of THUMOS14 dataset using two-stream I3D [7] pre-trained on Kinetics [24]. 16 consecutive frames are fed into I3D pre-trained network with a sliding window of stride 4. The extracted feature is collected after the last fully con- nected layer and has 1024-D feature space. After that, the two-stream features are further concatenated (2048-D) and utilized as the input of the model. Results. We compare the performa
143 ---
144 powerful GPU than the 1080 Ti. In comparison to early works, including both one-stage and two-stage methods, and those utilizing long-term TCM, TemporalMaxer achieves superior performance in both mAP and inference speed. frame input sequence with a stride of 16 to generate a set of 2304-D features. These features were then fed as input to our model. Result.

Tab. 2 shows our results. TemporalMaxer demonstrates notable performance on the EPIC-Kitchens 100 dataset, achieving an average mAP of 24.5% and
145 ---
146 e Extraction. We only utilize RGB stream as in- put for I3D network [7] pre-trained on Kinetics [24], fol- lowing [47], to extract features for MultiTHUMOS. The I3D pre-trained network is fed with 16 sequential frames through a sliding window with a stride of 4. The feature is extracted from the ?nal fully connected layer, resulting in a 1024-D feature space that serves as input for the model. Result. Tab. 4 provides a comparison of our perfor- Method tIoU 0.2 0.5 0.7 Avg PDAN [10] ? ? ? 17.3 MLAD [48
147 ---
148 lead to over?tting and thus less gen- eralization. To further concrete our thought, we replace the Transformer with a parameter-free operation, subsampling technique, in which features at even indexes are kept and odd indexes are removed to simulate stride 2 of TCM. Sur- prisingly, this none-parameter operation achieves higher re- sults than the convolution layer, shown in the second row of Tab. 5. This ?nding proves that TCM block may not need to contain many parameters, and the features from pretrai
149 ---
150 ative and is the potential for TAL. However, subsampling is prone to losing the most cru- cial information as only half of the video clip embeddings are kept after a TCM block. Thus, we replace Transformer with Average Pooling with kernel size 3 and stride 2. As expected, Average Pooling improves the result by 2.2 % mAP on average compared with the subsampling. It is because this operation does not drop any clip embedding which helps to retain the crucial information. However, the Average Pooling aver
151

13. assistant (2026-06-16T03:25:20.078Z)

I now have complete primary-source evidence for every component of the claim. Let me verify each part against the paper:

Claim breakdown vs. paper:

1. **"single-stage TAL detector built directly on the ActionFormer anchor-free head"** ? Confirmed (line 80): "Our approach, TemporalMaxer, belongs to the single-stage TAL model that utilizes a state-of-the-art ActionFormer as the baseline." Line 90: "We follow the anchor-free single-stage representation [28, 60]."

2. **"same sequence-labeling decoder where every moment is classified plus onset/offset regression"** ? Confirmed (line 80): "Similar to ActionFormer, TemporalMaxer follows a minimalistic design of sequence labeling where every moment is classified, and their corresponding action boundaries are regressed." Line 82: "regressed the onset and offset based on the current time step." Line 71: predicts (os_t, oe_t, ct).

3. **"via Focal Loss + DIoU loss"** ? Confirmed (line 71/76): "Following the baseline [60], the Focal Loss [31] and DIoU loss [64] are employed to supervise classification and regression outputs respectively."

4. **"ONLY change... self-attention temporal-context-modeling block replaced by a parameter-free MaxPooling block"** ? Confirmed (line 60): "The sole variation in our study was the substitution of the Transformer block in ActionFormer with the Max Pooling block." The Transformer block is ActionFormer's TCM (self-attention) ? confirmed line 108: "ActionFormer employs Transformer as a TCM block."

5. **"kernel 3, stride 2"** ? Confirmed: kernel size 3 (line 60/116: "kernel size of 3 for all TCM blocks"), stride 2 (line 108/140: "Max Pooling and employed with stride 2").

6. **"between feature-pyramid levels"** ? Confirmed (line 17/98): "Max Pooling as a Temporal Context Modeling block applied between temporal feature pyramid levels."

Every single component of the claim is directly and exactly supported by the primary source. One minor nuance: the kernel-3/stride-2 is the default configuration (kernel is ablated 3-6, but 3 is the default used in all main experiments), which the claim accurately describes. The claim is precise and not an overreach.

14. user (2026-06-16T03:25:27.711Z)

Structured output provided successfully

15. assistant (2026-06-16T03:25:29.908Z)

Verification complete. The claim is fully supported by the primary source and survives adversarial scrutiny ? refuted=false with high confidence.